From John McCarthy 1958

COMPUTATION CENTER

MASSACHUSETTS INSTITUTE OF TECHNOLOGY
CAMBRIDGE 39, MASSACHUSETTS

September 16, 1958

Dr. Werner Buchholz
Smith Road Laboratory
IBM Corporation
Poughkeepsie, New York

Dear Werner:

Thanks very much for the 7070 manual. I put several evenings into reading it and feel that I understand it quite well now. I think it will be a great machine.

As you know, one of my main interests is automatic programming languages and the computers which translate them into machine language. I read the 7070 manual with the purpose of seeing how the statements of a powerful algebraic programming language (such as the proposed international algebraic language and some languages I have in mind) would translate into 7070 programs and how smoothly and rapidly the 7070 would perform the translation. In general, it seemed to me that the 7070 would be far superior to present machines in both respects. However, there were a number of difficult points of which two seem to be of very great importance. They are:

1. It is important to be able to get an arbitrary field into the value field of an index register as quickly as possible, preferably without disturbing the accumulator. Why? When a field represents non-numeric data (e.g., a character of the text of a program or other text) the most important operation on it is table look-up or else a branch to a subroutine depending on it. Either of these requires getting the character into an index register.

Let me make a general remark at this point. The main value of index registers is for picking out elements of arrays. In matrix work one goes through the arrays systematically and the indexing system of the 7070 is admirably suited for this. However, in logical work, the index value is usually computed (most often by table look-up) and the next reference to the array requires a separate computation. For this kind of programming, the count and redefil fields of the 7070 are irrelevant. Nevertheless the 7070 is more powerful than the 704 for this purpose because the address of the load index or more precisely the load index instruction is indexable. If the storage requirements permit full or half words for each element nothing more can be asked for, and this

condition can easily be achieved at the cost of inefficient use of memory. A compiler on the 7000 would probably proceed most efficiently by first stretching out the input text one character per half word.

In the 7000 an arbitrary memory field can be put into an index register in two instructions, a numeric (full word) and a load value (half word). This could be reduced by any of the following devices:

1) Make the accumulator an index register. Then we only need the connect instruction. This has other advantages which may outweigh the disadvantage of reducing the number of ordinary index registers from 15 to 14.

2) Introduce a full word instruction that loads an index register from an arbitrary field. The necessary parts of this instruction are a) an address field (34 bits), b) an index modifying this address (4 bits), c) a length field specifying the length of the field to be loaded (6 bits, actually 5 bits is enough) and d) the index to be loaded (4 bits). Other 7000 full word features are optional.

3) Since the count and refill fields are irrelevant in their usual meaning in this kind of indexing (we may call it random access indexing) we may use them for another purpose so as to get a half word instruction. The instruction has the format of a direct index arithmetic instruction (see 3.4.2 in the manual). A and I designate the beginning of the field loaded and J the register loaded. The restriction of A to 18 bits is not a hardship since index register I has a 20 bit value field. The length of the field to be loaded is in the count field of J.

The last two procedures do not disturb the accumulator. The last one would be the best.

2. The second difficulty arises in connection with the indicator register. One of the best features of the 7000 system is the separation of testing and branching. This permits a test to be made when the data for it are most conveniently available, while the branch on the result of the test is made later. Of course, since the calculations between the branch and the test may affect the indicator register it is frequently necessary to store the result of the test. The straightforward way of doing this is to use a transmit instruction to store the whole indicator register and then use a branch on bit instruction. Unfortunately, this facility does not directly fill the needs of a programming language like the proposed international algebraic language. In this language (and I think this feature will be included in all future algebraic languages) one-bit "propositional" variables are provided for and statements like

$$p \leftarrow (b=h)$$

are allowed. This means that the one-bit variable is to be set equal to 1 if a=b and to 0 otherwise. Now the variable p must be assigned a

fixed storage location (say bit 233.17 in 7000S notation). If in one place in the program we have

p ← (a-b)

and in another have

p ← (c-d)

we cannot get the indicator bit into the right place without a shift in one case or the other. What is required is an instruction that will transmit a single bit from an arbitrary place in the indicator register to an arbitrary place in memory (actually it would be adequate to be able to get just the first bit in a word). The former would require a full word instruction while the latter could be done by an instruction of the same format as a branch on indicator instruction which would store the indicator bit in either bit 0 or bit 35 of a word.

In general, the 7000S like all other computers is weak on using logical combinations of test results as branch conditions. The above mentioned instruction by allowing propositional quantities (results of tests) to be lined up would facilitate logical operations on them.

In closing, I will make one comment on the inquiry station. For very close interaction between man and machine, there should be an option in which each key struck by the man interrupts the calculation (without the signal key being used). This would permit, for example, a programmer to sit at a typewriter and code directly into the machine with the machine complaining about any ungrammatical character as soon as it was produced.

The difficulties mentioned above are perhaps minor, but I think they will be noticed. Of course, I may have overlooked features which obviate them.

I will repeat what I said at the beginning - that I was enormously impressed by the elegance of the system, and I think programmers will find it a great advance.

Best regards

John McCarthy

JMc:gm

P.S. I assume you anticipated some kibitzing when you gave me a copy of the manual! Thanks again.